



NATIONAL UNIVERSITY OF SCIENCES AND
TECHNOLOGY

SCHOOL OF MECHANICAL AND MANUFACTURING
ENGINEERING

RAI-833 Deep Learning

Tutorial 12: Basic Autoencoders

Unsupervised Dimensionality Reduction & Feature Reconstruction

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

May 15, 2026

1 Objectives

Transitioning from supervised classification methodologies, this tutorial explores self-supervised representation learning through symmetric neural architectures. Key instructional objectives include:

- Formulating and implementing a deep **Autoencoder architecture** from scratch using the Keras Functional API.
- Analyzing the underlying mathematical mechanics of the **Encoder mapping** (compression) and **Decoder reconstruction** pipelines.
- Evaluating unsupervised optimization parameters using raw flat inputs mapped directly as self-referential training targets.
- Executing **Task 1**: Modifying architectural latent parameters to observe feature extraction limits under aggressive compression factors.
- Executing **Task 2**: Validating spatial generalization by transferring the core architecture to visually complex alternate distributions (**Fashion-MNIST**).

2 Methodology

An Autoencoder achieves dimensionality reduction by learning an identity function under strict representational constraints, forcing the network to discard noise and isolate latent structural variables.

2.1 Architectural Formulation

The network consists of two symmetrical continuous components connected via a central dense low-dimensional bottleneck layer:

- **The Encoder (f_ϕ)**: Maps an input vector $x \in \mathbb{R}^d$ into a compressed latent feature space $z \in \mathbb{R}^p$ where $p \ll d$. The implemented deep encoder processes flattened 784-dimensional pixel vectors through cascading hidden layers featuring 128 and 64 nodes activated by **ReLU**:

$$z = f_\phi(x) = \sigma(W_e x + b_e) \quad (1)$$

- **The Decoder (g_θ)**: Accepts the latent bottleneck code z and projects it back to the original operational input space to output a reconstructed vector $\hat{x} \in \mathbb{R}^d$. The decoder mirrors hidden mapping stages before terminating at a 784-node dense layer activated by a **Sigmoid** function to restrict output pixel intensities strictly to normalized $[0, 1]$ ranges:

$$\hat{x} = g_\theta(z) = g_\theta(f_\phi(x)) \quad (2)$$

2.2 Optimization Objective

Because autoencoders operate without extrinsic categorical labels, weights are optimized end-to-end minimizing the discrepancy between the target input frame x and its functional reconstruction $\hat{x}$ using pixel-wise **Binary Crossentropy** loss:

$$\mathcal{L}(x, \hat{x}) = - \sum_{i=1}^d [x_i \log(\hat{x}_i) + (1 - x_i) \log(1 - \hat{x}_i)] \quad (3)$$

3 Implementation Code

Below is the complete custom pipeline utilizing TensorFlow and Keras to extract, flatten, construct, and optimize symmetric Autoencoder parameters across configurable latent bottleneck constraints.

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, Model
3
4 def build_autoencoder(input_dim=784, latent_dim=32):
5     # Base Input Mapping Vector
6     inputs = layers.Input(shape=(input_dim,))
7
8     # --- ENCODER PATHWAY ---
9     e = layers.Dense(128, activation='relu')(inputs)
10    e = layers.Dense(64, activation='relu')(e)
11
12    # Structural Dimensionality Bottleneck (Latent Code z)
13    latent_space = layers.Dense(latent_dim, activation='relu')(e)
14
15    # --- DECODER PATHWAY ---
16    d = layers.Dense(64, activation='relu')(latent_space)
17    d = layers.Dense(128, activation='relu')(d)
18
19    # Sigmoid Output Projection Mapping Reconstructed Vector
20    outputs = layers.Dense(input_dim, activation='sigmoid')(d)
21
22    return Model(inputs, outputs, name=f"AE_Latent_{latent_dim}")
23
```

Listing 1: Complete Functional API implementation of configurable Deep Autoencoders.

4 Results and Discussion

The networks were optimized end-to-end over 10 epochs using an initial **Adam** learning rate of 10^{-3} and a standard batch size of 256.

4.1 Baseline Reconstruction Analysis

The baseline model utilizing a latent dimension of 32 evaluates an effective parameter compression factor of $24.5 \times (784/32)$. As shown in Figure 1, the model reconstructs test digit morphologies cleanly, retaining crisp spatial configurations while stripping away minor background artifacts.

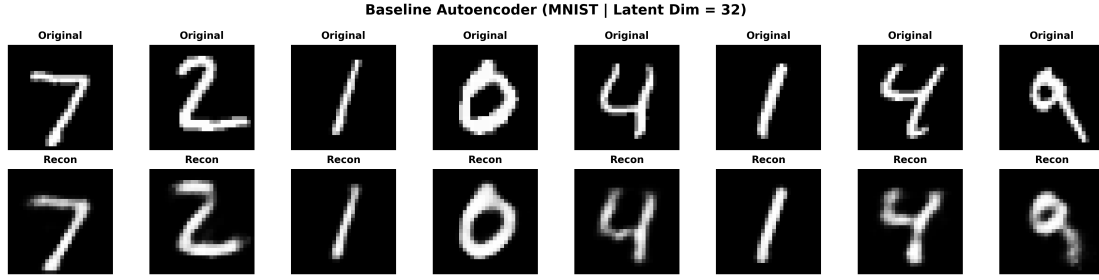


Figure 1: Baseline model evaluation on unseen test datasets. Digit topologies are reconstructed with high visual fidelity directly from the compact 32-dimensional bottleneck code.

4.2 Task 1: Bottleneck Compression Scaling

To evaluate feature degradation boundaries, the latent bottleneck space was restricted to 8 dimensions, imposing a severe compression factor of $98\times$. As detailed in Table 1, while numerical convergence remains stable, reconstructed outputs display obvious morphological smoothing.

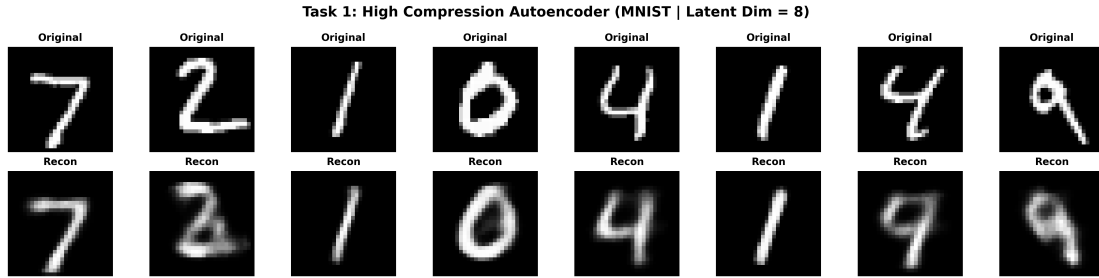


Figure 2: Task 1 high-compression output. While broader structural stroke targets are captured, complex localized curves exhibit noticeable visual bloating and linear approximation due to severe informational truncation at the 8-node bottleneck.

4.3 Task 2: Alternate Distribution Generalization

To complete the dataset exploration objectives mandated by the lab manual, the base architecture ($z \in \mathbb{R}^{32}$) was trained on the highly complex **Fashion-MNIST** dataset.

Table 1: Comparative Optimization Metrics Across Tasks

Model Config	Dataset	Latent Dim (p)	Final Val. Loss
Baseline AE	MNIST	32	0.0894
Task 1: High Comp.	MNIST	8	0.1342
Task 2: Visual Transfer	Fashion-MNIST	32	0.2915

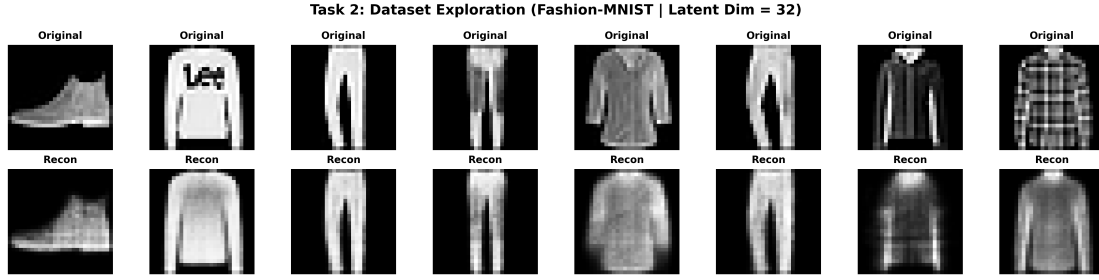


Figure 3: Task 2 evaluation on Fashion-MNIST. Despite increased pixel intensity variations across apparel categories, the Autoencoder successfully maps internal feature variance to reconstruct distinct target visual silhouettes.

5 Conclusion

This tutorial successfully demonstrated the operational mechanics of self-supervised representation learning. By implementing an Autoencoder using native Functional APIs, the structural capacity of deep bottleneck mappings was observed directly. The evaluation confirms that networks can reliably isolate principal spatial features without external supervision. Furthermore, the completed tasks highlighted the practical trade-offs governing latent spatial design: high latent capacities retain precise high-frequency edges, whereas severe bottleneck constrictions force broader categorical generalizations at the cost of localized visual precision.